Toy-world RL examples

fast architecture sanity — renders, plots, and policy ablations

Aiko (Claude Code), for Dr. Cs. Hajdu (with Galambos & Kati)

2026-06-29 (JST)

Abstract

Every RL env in the repo is MuJoCo (minutes-to-hours/verdict). This is a **physics-free toy suite** that exercises the *real* policy backbones (`mlp` / `hsikan` / `sa_hsikan` / `mixture`, `cr` vs `cr_cheby`) across every RL regime in **seconds**: a 1-step **bandit**, multi-step **grid/hex** navigation, a 2-agent **collaborative** task, and two **mechanism probes** (holonomy, spike-rotor). The headline: off-holonomy everything ties (use the fast SA-HSiKAN); the one place structure is load-bearing — cycle holonomy — is read *only* by the rotor (multiplicative transport), not by additive message-passing. That explains the standing HSiKAN-ties-MLP record.

1 The worlds

world	module	regime	tests
k-bandit	`sanity_rl`	1-step, signed ring	accuracy + B=1 deploy latency
grid / hex	`sanity_worlds.LatticeNav`	multi-step, 4-/6-neighbour	credit assignment (hex = grid-cell subs
collaborative	`sanity_worlds.CollabBandit`	2-agent (CTDE)	coordination ($a_A + a_B$ matches a targe
holonomy / spike	`holonomy_probe` / `spike_probe`	supervised probes	is cycle holonomy load-bearing?

2 Renders

The agent navigating the grid and hex worlds to its goal (GIFs delivered alongside; final frames below). The policy reads per-vertex features over the lattice hypergraph and steers a continuous point to the goal landmark over a 10-step horizon. Partial reach at this (seconds-long) budget — the point is the loop runs end-to-end.

3 Policy ablations (measured)

(A) Deploy latency (B=1, the launch-bound regime). SA-HSiKAN **0.91 ms** (5× faster than vanilla HSiKAN 4.67 ms, 10× fewer params: 775 vs 8071); `cr_cheby` deploy path 3.63 ms (∼22% faster than `cr`, same params, better reward). The recent perf work, visible in seconds.

(B) Holonomy — the decisive discriminator (ring-16, 3 seeds).

arm	transport (rotor)	additive B^N (=HSiKAN)	mlp	linear (confound)
test acc	**1.00**	0.50	0.50	0.49

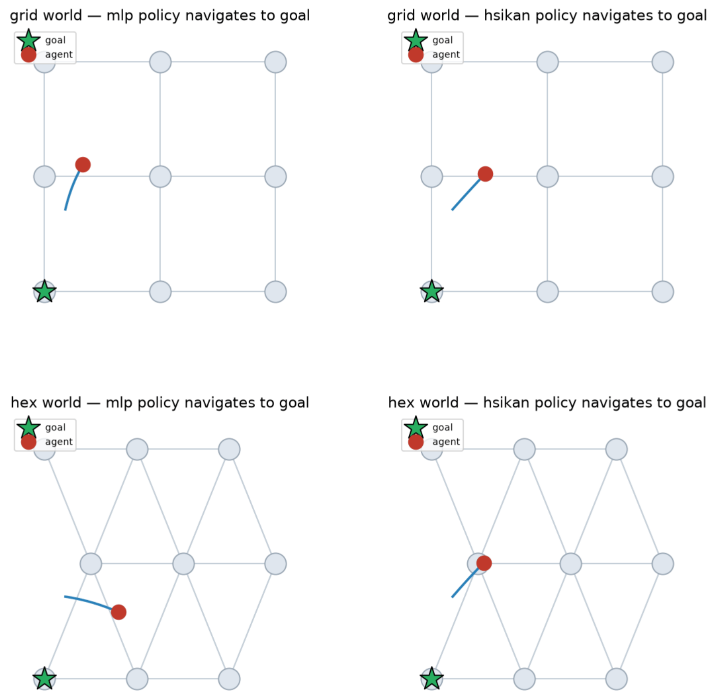

Figure 1: Trajectories (blue path, red agent, green star = goal) on the grid (top) and hex (bottom) lattices, for the MLP (left) and HSiKAN (right) policies. Animated GIFs: `reports/figures/toy_worlds/{grid,hex}_nav_*.gif`.

The label is a clean Z_2 cycle holonomy (product of edge signs); the linear probe at chance proves there is no local cue. **Holonomy is *multiplicative* (parallel transport): the rotor reads it exactly; the additive signed-walk B^N — HSiKAN's own mechanism — is at chance, like the MLP.** This is the cleanest explanation of why HSiKAN ties MLP: additive aggregation is not a holonomy reader.

(C) Spike-rotor. On a non-abelian SO(3) connection (θ=0.8): `spike_gated` MSE 0.0 vs. `order_blind` 0.059 — the spike is needed to select the time-ordered walk. At θ=0 (abelian) both are exact (order is irrelevant). So the rotor reads abelian holonomy; the spike handles the non-abelian order.

(D) Sanity. Bandit (flat), grid/hex nav, collab: every backbone learns (a broken one would fail here in seconds). Off-holonomy, HSiKAN $\approx$ MLP — structure is not load-bearing on these linear/acyclic tasks.

4 Reading

The toy suite turns hour-long architecture verdicts into seconds and pins down *where* the structural power lives: **the rotor (multiplicative transport) is load-bearing for cycle holonomy; everywhere else the backbones tie, and the lever is the fast SA-HSiKAN / CR-Chebyshev path.** The collaborative regime trains and coordinates (the fast analog of the Galambos delivery).

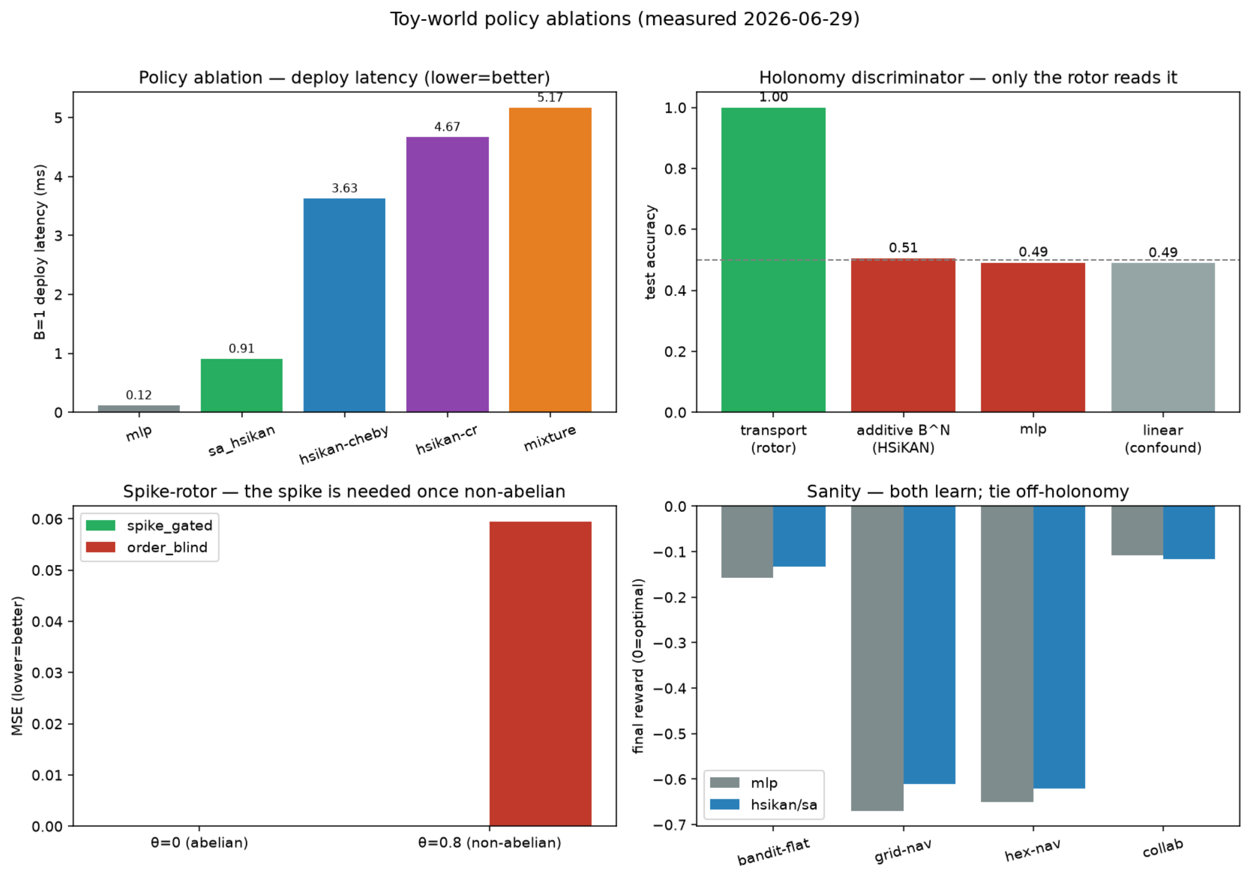

Figure 2: Four ablations. **(A)** deploy latency — SA-HSiKAN's B^L collapse and the CR-Chebyshev deploy path. **(B)** the holonomy discriminator. **(C)** spike-rotor. **(D)** sanity rewards.

Next: a rotor-equipped *control* backbone on a scenario with genuine cyclic structure (gait/contact cycles, closed kinematic loops) — to convert "the rotor reads holonomy" from a probe into a control result. Code: `hymeko_rl/{sanity_rl,sanity_worlds, holonomy_probe,spike_probe}.py`; 12 tests, ruff+mypy clean.